

Day 14 - 02th July 2025

Task 1:

Create a custom node, add elements to it and traverse it..

```
public class CustomLinkedList {  
  
    // Node class (inner class)  
    static class Node {  
        int data;  
        Node next;  
  
        Node(int data) {  
            this.data = data;  
            this.next = null;  
        }  
    }  
  
    // Head of the list  
    Node head = null;  
  
    // Method to add elements to the list  
    public void add(int data) {  
        Node newNode = new Node(data);  
  
        if (head == null) {  
            head = newNode; // First node  
        } else {  
            Node temp = head;  
            while (temp.next != null) {  
                temp = temp.next; // Go to last node  
            }  
            temp.next = newNode;  
        }  
    }  
}
```

```
public void add(int data) { 3 usages
    Node temp = head;
    while (temp.next != null) {
        temp = temp.next; // Go to last node
    }
    temp.next = newNode; // Add at the end
}

// Method to traverse and print the list
public void traverse() { 1 usage
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}

// Main method
public static void main(String[] args) {
    CustomLinkedList list = new CustomLinkedList();

    list.add(10);
    list.add(20);
    list.add(30);
}
```

Activate Win

```

        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }
        System.out.println("null");
    }

    // Main method
    public static void main(String[] args) {
        CustomLinkedList list = new CustomLinkedList();

        list.add(10);
        list.add(20);
        list.add(30);

        System.out.println("Traversing the Linked List:");
        list.traverse();
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editi
Traversing the Linked List:
10 -> 20 -> 30 -> null

Process finished with exit code 0

```

Task 2 👍

What do you understand by traversing elements in a linked list.

A **linked list** is a data structure made up of nodes, where each node contains data and a reference (or pointer) to the next node in the list. Unlike arrays, linked lists do not allow random access by index because their elements are stored non-contiguously in memory.

Task 3:

Create a Circular Linked list using Task 1 Singly linked list.

```
main.java CircularLinkedList.java
public class CircularLinkedList {

    // Node class to represent each element
    static class Node { 6 usages
        int data; 2 usages
        Node next; 7 usages

        Node(int data) { 1 usage
            this.data = data;
            this.next = null;
        }
    }

    // Head of the Circular Linked List
    Node head = null; 9 usages

    // Method to add a new node to the list
    public void add(int data) { 3 usages
        Node newNode = new Node(data);

        if (head == null) {
            // If the list is empty, the new node is both head and tail
            head = newNode;
            newNode.next = head; // Point to itself (circular reference)
        } else {
            // Find the last node (node whose next is head)
            Node temp = head;
            while (temp.next != head) {
                temp = temp.next;
            }
            temp.next = newNode;
        }
    }
}
```

Activate W

```

public void add(int data) { 3 usages
    newNode.next = head; // Point to itself (circular reference)
} else {
    // Find the last node (node whose next is head)
    Node temp = head;
    while (temp.next != head) {
        temp = temp.next;
    }

    // Add new node after the last node and make it circular
    temp.next = newNode;
    newNode.next = head; // Points back to head, making it circular
}
}

```

// Method to display the contents of the Circular Linked List

```

public void display() { 1 usage
    if (head == null) {
        System.out.println("List is empty.");
        return;
    }

```

```

    Node temp = head;
    do {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    } while (temp != head);
}

```

Activate Windows

68:1 C:\Program Files\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.29.30133\bin\Hostx64\x64\cl.exe

```

}

// Method to display the contents of the Circular Linked List
public void display() { // 1 usage
    if (head == null) {
        System.out.println("List is empty.");
        return;
    }

    Node temp = head;
    do {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    } while (temp != head); // Stop when we loop back to the head

    System.out.println("(back to head)");
}

// Main method to test Circular Linked List
public static void main(String[] args) {
    CircularLinkedList list = new CircularLinkedList();

    // Adding some elements to the circular linked list
    list.add(10);
    list.add(20);
}

```

Active

Task 4 :

Stacks

Create a code to implement a stack

12.42 – plz complete..

```

        System.out.println("(back to head)");
    }

    // Main method to test Circular Linked List
    public static void main(String[] args) {
        CircularLinkedList list = new CircularLinkedList();

        // Adding some elements to the circular linked list
        list.add(10);
        list.add(20);
        list.add(30);

        // Displaying the circular linked list
        System.out.println("Circular Linked List:");
        list.display();
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java
Circular Linked List:
10 -> 20 -> 30 -> (back to head)

Process finished with exit code 0

```

```

import java.util.Stack;

> class Ds_Stack_Pop {
>     public static void main(String[] args) {
        // Create a stack of String type
        Stack<String> names = new Stack<>();

        // Push elements onto the stack
        names.push(item: "Prasunamba");
        names.push(item: "Meher");
        names.push(item: ".MK");

        // Print the stack before popping
        System.out.println("Before deletion:");
        System.out.println("Stack of names: " + names);

        // Pop the top element from the stack
        String poppedElement = names.pop();

        // Print the stack after popping and the popped element
        System.out.println("After deletion:");
        System.out.println("Stack of names: " + names);
        System.out.println("Deleted element is: " + poppedElement);
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe"
Before deletion:
Stack of names: [Prasunamba, Meher, .MK]
After deletion:
Stack of names: [Prasunamba, Meher]
Deleted element is: .MK

```

Task 5:

Create a stack and pop the element also print the popped element.

12.50 plz complete

```
import java.util.Stack;
class Ds_Stack_Pop {
    public static void main(String[] args) {
        Stack<String> names= new Stack<>();

        names.push("Prasunamba");
        names.push("Meher");
        names.push(".MK");

        System.out.println("before deletion ");
        System.out.println("Stack of names: " + names);

        System.out.println("after deletion ");
        String dummy = names.pop();

        System.out.println("Stack of names: " + names);
        System.out.println("deleted element is "+ dummy);
    }
}
```

```

import java.util.Stack;

class Ds_Stack_Pop {
    public static void main(String[] args) {
        Stack<String> names = new Stack<>();

        // Push elements to the stack
        names.push(item: "Prasunamba");
        names.push(item: "Meher");
        names.push(item: ".MK");

        System.out.println("Before deletion:");
        System.out.println("Stack of names: " + names);

        // Pop element from the stack
        String dummy = names.pop();

        System.out.println("After deletion:");
        System.out.println("Stack of names: " + names);
        System.out.println("Deleted element is: " + dummy);
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:
Before deletion:
Stack of names: [Prasunamba, Meher, .MK]
After deletion:
Stack of names: [Prasunamba, Meher]
Deleted element is: .MK

Process finished with exit code 0

```

> src > © Ds Stack Pop

Task 6:

Find an element in the stack and display the position

Hint 👍

```
Int position = names.search("value");
```

```
import java.util.Stack;
class Ds_Stack_SearchPosition {
    public static void main(String[] args) {
        Stack<String> names= new Stack<>();

        names.push("Prasunamba");
        names.push("Meher");
        names.push(".MK");

        System.out.println("Stack of names: " + names);

        String Val = "Meher";
        int position = names.search(Val);

        System.out.println("the searched value is at position " + position);
    }
}
```

```
Main.java  Ds_Stack_SearchPosition.java x
import java.util.Stack;

class Ds_Stack_SearchPosition {
    public static void main(String[] args) {
        Stack<String> names = new Stack<>();

        // Pushing values into the stack
        names.push(item: "Prasunamba");
        names.push(item: "Meher");
        names.push(item: ".MK");

        // Displaying stack
        System.out.println("Stack of names: " + names);

        // Value to search
        String val = "Meher";

        // Search the value in the stack
        int position = names.search(val);

        // Display result
        if (position == -1) {
            System.out.println("The value '" + val + "' was not found in the stack.");
        } else {
            System.out.println("The searched value '" + val + "' is at position " + position + " from the top of the stack.");
        }
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBr
Stack of names: [Prasunamba, Meher, .MK]
The searched value 'Meher' is at position 2 from the top of the stack.
|
Process finished with exit code 0
```

Task 7:

Peek the element and print it ..

```
import java.util.Stack;
class Ds_Stack_Peek {
    public static void main(String[] args) {
        Stack<String> names= new Stack<>();
```

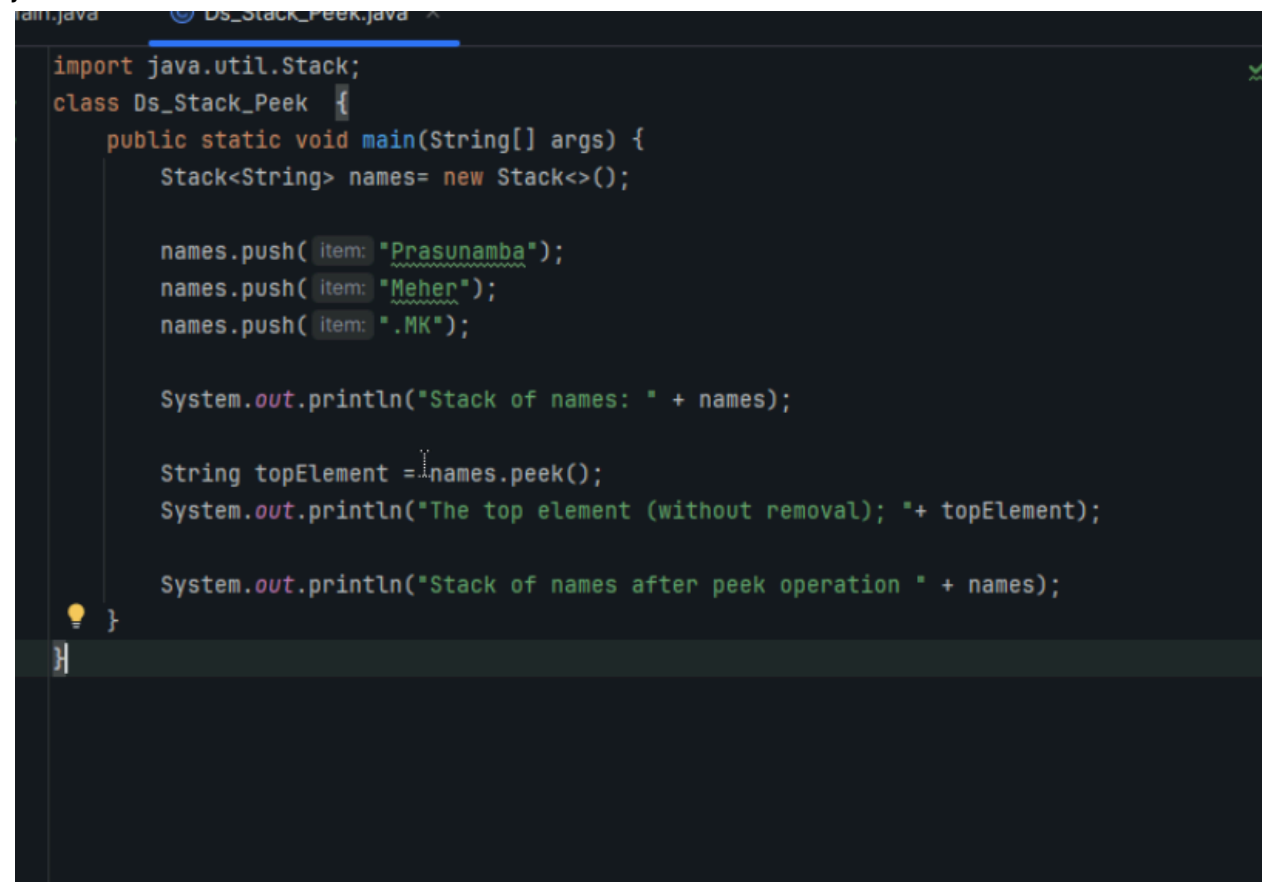
```
names.push("Prasunamba");
names.push("Meher");
names.push(".MK");
```

```
System.out.println("Stack of names: " + names);
```

```
String topElement = names.peek();
System.out.println("The top element (without removal); "+ topElement);
```

```
System.out.println("Stack of names after peek operation " + names);
```

```
}
```

A screenshot of an IDE window titled "Ds_Stack_Peek.java". The code is as follows:

```
import java.util.Stack;
class Ds_Stack_Peek {
    public static void main(String[] args) {
        Stack<String> names= new Stack<>();

        names.push( item: "Prasunamba");
        names.push( item: "Meher");
        names.push( item: ".MK");

        System.out.println("Stack of names: " + names);

        String topElement = names.peek();
        System.out.println("The top element (without removal); "+ topElement);

        System.out.println("Stack of names after peek operation " + names);
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Progr
Stack of names: [Prasunamba, Meher, .MK]
The top element (without removal); .MK
Stack of names after peek operation [Prasunamba, Meher, .MK]

Process finished with exit code 0
|
```

Task 8:

Check if the stack is empty or not?

```
import java.util.Stack;
```

```
public class Ds_Stack_Empty {
    public static void main(String[] args) {
        Stack<String> names = new Stack<>();
        System.out.println("Is the stack empty? " + names.empty());

        names.push("Prasunamba");
        names.push("Meher");

        System.out.println("Is the stack empty? " + names.empty());

        while (!names.empty()){
            System.out.println("Popped: " + names.pop());
        }

        System.out.println("Is the stack empty? " + names.empty());
    }
}
```

```
1 import java.util.Stack;
2
3 public class Ds_Stack_Empty {
4     public static void main(String[] args) {
5         Stack<String> names = new Stack<>();
6         System.out.println("Is the stack empty? " + names.empty());
7
8         names.push("Prasunamba");
9         names.push("Meher");
10
11        System.out.println("Is the stack empty? " + names.empty());
12
13        while (!names.empty()){
14            System.out.println("Popped: " + names.pop());
15        }
16
17        System.out.println("Is the stack empty? " + names.empty());
18    }
19 }
```

Run Ds_Stack_Empty

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar" ...

Is the stack empty? true
Is the stack empty? false
Popped: Meher
Popped: Prasunamba
Is the stack empty? true
Process finished with exit code 0

Task 9:

What are the methods of the stack class

The **Stack** class in Java (part of `java.util` package) represents a last-in-first-out (LIFO) stack of objects. It extends `Vector` and provides standard stack operations.

push(E item)
pop()
peek()
empty()
search(Object o)

Task 10:

What are the common operations in Queues

A **Queue** is a **FIFO** (First-In-First-Out) data structure where elements are added at the **rear** (tail) and removed from the **front** (head).

Enqueue
Dequeue
Peek

isEmpty

Size

Task 11:

Wap to create a queue with custom methods

Is empty ()

Is full()

Enque

Deque

Peek

display()

```

> public class Queue1 {
    int queueLength = 3; 2 usages
    int items[] = new int[queueLength]; 4 usages
    int front = -1; 8 usages
    int back = -1; 10 usages

    void enqueue(int itemValue) { no usages
        if (isFull()) {
            System.out.println("Queue is full");
        } else if (front == -1 && back == -1) {
            front = back = 0;
            items[back] = itemValue;
        } else {
            back++;
            items[back] = itemValue;
        }
    }

    void dequeue() { no usages
        if (isEmpty()) {
            System.out.println("Queue is empty. Nothing to dequeue");
        } else if (front == back) {
            front = back = -1;
        } else {
            front++;
        }
    }

    void display() { no usages
        if (isEmpty()) {
            System.out.println("Queue is empty");
        }
    }
}

```

```
void display() { no usages
    if (isEmpty()) {
        System.out.println("Queue is empty");
    } else {
        System.out.print("Queue elements: ");
        for (int i = front; i <= back; i++) {
            System.out.print(items[i] + " ");
        }
        System.out.println();
    }
}
```

```
boolean isFull() { 1 usage
    return back == queueLength - 1;
}
```

```
boolean isEmpty() { 3 usages
    return front == -1 && back == -1;
}
```

```
void peek() { no usages
    if (isEmpty()) {
        System.out.println("Queue is empty. Nothing to peek.");
    } else {
        System.out.println("Front value is: " + items[front]);
    }
}
```

```
void peek() { no usages
```

```
    System.out.println("Queue is empty. Nothing to peek.");
```

```
  } else {
```

```
    System.out.println("Front value is: " + items[front]);
```

```
  }
```

```
}
```



```
public static void main(String[] args) {
```

```
    nameQueue1 myQueue = new nameQueue1();
```

```
    myQueue.enqueue(111);
```

```
    myQueue.enqueue(222);
```

```
    myQueue.enqueue(777);
```

```
    myQueue.enqueue(999); // Will show "Queue is full"
```

```
    myQueue.display();
```

```
    myQueue.peek();
```

```
    myQueue.dequeue();
```

```
    myQueue.display();
```

```
    myQueue.peek();
```

```
}
```

```
}
```

```
"C:\Program Files\Java\jdk-17\bin\java
```

```
Queue is full
```

```
Queue elements: 111 222 777
```

```
Front value is: 111
```

```
Queue elements: 222 777
```

```
Front value is: 222
```

```
Process finished with exit code 0
```

Home Tasks:

1. Write a recursive function to search for an element in an array

```
1 public class RecursiveArraySearch {
4     public static int search(int[] arr, int target, int index) { 2 usages
9
10         // If current element matches the target
11         if (arr[index] == target) {
12             return index; // return the position (0-based)
13         }
14
15         // Recursive call for next index
16         return search(arr, target, index + 1);
17     }
18
19     public static void main(String[] args) {
20         int[] myArray = {10, 25, 30, 45, 50};
21         int target = 45;
22
23         int result = search(myArray, target, 0);
24
25         if (result == -1) {
26             System.out.println("Element not found in the array.");
27         } else {
28             System.out.println("Element found at index: " + result);
29         }
30     }
31 }
32
```

- 2.

Write a recursive function to count the digits of a positive integer (do also for sum of digits)

```

1 public class DigitOperations {
2     public static int countDigits(int num) { 2 usages
3         return 1 + countDigits(num / 10);
4     }
5
6     // Recursive function to sum digits
7     public static int sumDigits(int num) { 2 usages
8         // Base case
9         if (num == 0)
10            return 0;
11        // Recursive case: get last digit and sum recursively
12        return (num % 10) + sumDigits(num / 10);
13    }
14
15    public static void main(String[] args) {
16        int number = 12345;
17
18        int digitCount = countDigits(number);
19        int digitSum = sumDigits(number);
20
21        System.out.println("Number: " + number);
22        System.out.println("Total digits: " + digitCount);
23        System.out.println("Sum of digits: " + digitSum);
24    }
25 }

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe"
Number: 12345
Total digits: 5
Sum of digits: 15

Process finished with exit code 0

```

Write a recursive function to reverse a null-terminated string

```
© Main.java  ReverseStringRecursive.java ×
1  public class ReverseStringRecursive {
4      public static void reverse(char[] str, int start, int end) { 2 usages
8          char temp = str[start];
9          str[start] = str[end];
10         str[end] = temp;
11
12         // Recursive call
13         reverse(str, start + 1, end - 1);
14     }
15
16     public static void main(String[] args) {
17         String input = "hello";
18
19         // Convert string to char array
20         char[] chars = input.toCharArray();
21
22         // Call recursive reverse
23         reverse(chars, 0, chars.length - 1);
24
25         // Print reversed string
26         String reversed = new String(chars);
27         System.out.println("Original: " + input);
28         System.out.println("Reversed: " + reversed);
29     }
30 }
31
```

```
"C:\Program Files\Java\jdk
Original: hello
Reversed: olleh

Process finished with exit
```

Write a recursive function to convert a decimal number to binary

```
Main.java  DecimalToBinaryRecursive.java x
public class DecimalToBinaryRecursive {
    public static void convertToBinary(int number) { 2 usages
        if (number == 0) {
            return; // Base case
        }

        // Recursive call for higher bits
        convertToBinary(number / 2);

        // Print current bit (after recursive call to maintain order)
        System.out.print(number % 2);
    }

    public static void main(String[] args) {
        int decimal = 13;

        System.out.print("Binary of " + decimal + " is: ");
        if (decimal == 0) {
            System.out.print(0); // Special case for 0
        } else {
            convertToBinary(decimal);
        }
        System.out.println(); // for newline
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe
Binary of 13 is: 1101

Process finished with exit code 0
```

Write a recursive function to check if a string is a palindrome or not

```
public class PalindromeRecursive {

    // Recursive method to check palindrome
    public static boolean isPalindrome(String str, int start, int end) { 2 usages
        // Base case: crossed pointers or single char
        if (start >= end) {
            return true;
        }

        // If mismatch found
        if (str.charAt(start) != str.charAt(end)) {
            return false;
        }

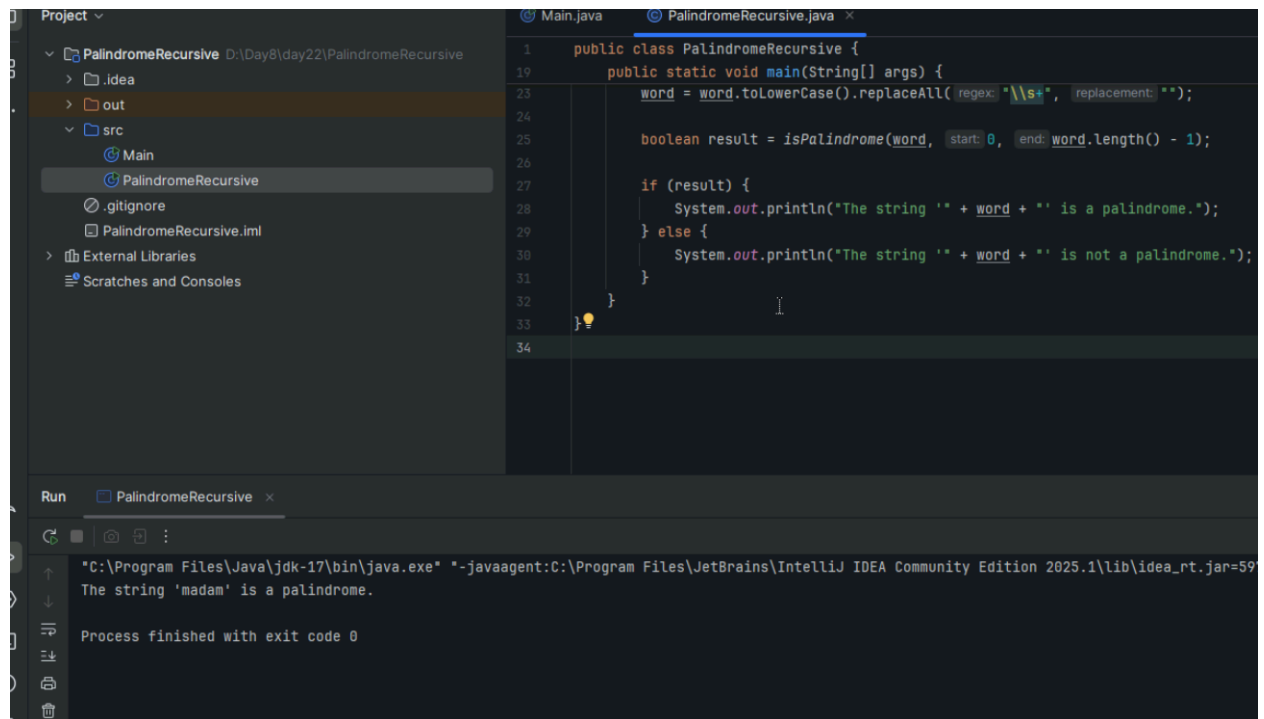
        // Recursive check for inner substring
        return isPalindrome(str, start: start + 1, end: end - 1);
    }

    public static void main(String[] args) {
        String word = "madam"; // Try other examples like "racecar", "hello", etc.

        // Normalize case and remove spaces if needed
        word = word.toLowerCase().replaceAll(regex: "\\s+", replacement: "");

        boolean result = isPalindrome(word, start: 0, end: word.length() - 1);

        if (result) {
            System.out.println("The string '" + word + "' is a palindrome.");
        } else {
            System.out.println("The string '" + word + "' is not a palindrome.");
        }
    }
}
```



Write a recursive function to copy one array to another

```
1 public class RecursiveArrayCopy {
2     // Runnable class
3     // Recursive function to copy array elements
4     @ public static void copyArray(int[] source, int[] destination, int index) { 2 usages
5         // Base case: when index reaches array length
6         if (index == source.length) {
7             return;
8         }
9
10        // Copy current element
11        destination[index] = source[index];
12
13        // Recursive call for next index
14        copyArray(source, destination, index: index + 1);
15    }
16
17    public static void main(String[] args) {
18        int[] source = {10, 20, 30, 40, 50};
19        int[] destination = new int[source.length];
20
21        copyArray(source, destination, index: 0);
22
23        // Display copied array
24        System.out.print("Copied array: ");
25        for (int val : destination) {
26            System.out.print(val + " ");
27        }
28    }
29 }
30
```

30:1 CRLF UTF-8

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

```
"C:\Program Files\Java\jdk-17\bin\java.exe"
Copied array: 10 20 30 40 50
Process finished with exit code 0
```

Wap to find the factorial of a number

The image shows a screenshot of an IDE with two main panels. The top panel displays the source code for a Java program named `FactorialProgram.java`. The code includes imports, class definition, and two methods: `factorialRecursive` and `factorialIterative`. The bottom panel shows the project structure on the left and the execution output at the bottom.

```
1 import java.util.Scanner;
2
3 public class FactorialProgram {
4
5     // Recursive method to find factorial
6     public static long factorialRecursive(int n) { 2 usages
7         if (n == 0 || n == 1)
8             return 1;
9         return n * factorialRecursive(n - 1);
10    }
11
12    // Iterative method to find factorial
13    public static long factorialIterative(int n) { 1 usage
14        long fact = 1;
15        for (int i = 2; i <= n; i++) {
16            fact *= i;
17        }
18        return fact;
19    }
20
21    public static void main(String[] args) {
22        Scanner scanner = new Scanner(System.in);
23
24        System.out.print("Enter a number to find its factorial: ");
25        int num = scanner.nextInt();
26
27        if (num < 0) {
28            System.out.println("Factorial is not defined for negative numbers.");
29        } else {
30            System.out.println("Recursive factorial of " + num + " is: " + factorialRecursive(num));
31            System.out.println("Iterative factorial of " + num + " is: " + factorialIterative(num));
32        }
33
34        scanner.close();
35    }
36 }
37
```

The bottom panel shows the project structure on the left with `src` containing `FactorialProgram`, `Main`, `.gitignore`, and `FactorialProgram.iml`. The execution output at the bottom shows the command used to run the program and the prompt "Enter a number to find its factorial:".

What is the difference between recursion and iteration

Recursion

Definition:

Recursion is a programming technique where a function **calls itself** to solve a problem by breaking it down into smaller subproblems.

Explanation:

The recursive function keeps calling itself with simpler inputs until it reaches a **base case** — a condition where it stops calling itself and starts returning results back up the chain. It's useful for problems that have a natural recursive structure, like calculating factorials or traversing trees.

Iteration

Definition:

Iteration is a technique where a block of code is **repeated multiple times** using loops such as `for`, `while`, or `do-while`.

Explanation:

Instead of calling itself, the program uses looping constructs to perform repetitive tasks until a condition is met. Iteration is often used when the number of repetitions is known or can be determined during execution, like processing items in a list.